

UNIVERSIDADE FEDERAL DO AMAZONAS (UFAM)

Centro de P&D em Tecnologia Eletrônica e da Informação — CETELI

Programa de Pós-Graduação em Engenharia Elétrica — PPGEE

FTL094 — Engenharia de Software de Tempo Real

Período Letivo 2026/1

Código-Fonte

Sistema de Navegação Autônoma para o Robô Móvel Khepera IV

Equipe

Gabriel Henrique de Souza Nazaré

Luan Alexandre Ramos Barreto

Matheus Rocha Canto

Matheus Serrão Uchôa

Ricardo Mendonça Braz

Yan Fernandes da Silva

Docente responsável: Prof. Dr. [nome do docente]

Controle de versões

Versão	Data	Autor	Descrição
1.0	9 de julho de 2026	Equipe FTL094	Emissão inicial do documento.

Sumário

1	Introdução	2
1.1	Objetivo	2
1.2	Escopo	2
2	Organização e estrutura do código	2
3	Compilação e execução	2
4	Listagens	3
4.1	Mundo de simulação — <code>worlds/first.wbt</code>	3
4.2	Controlador de navegação — <code>controllers/goto_avoid/goto_avoid.c</code>	4
4.3	Script de compilação — <code>controllers/goto_avoid/Makefile</code>	7
5	Correspondência código ↔ arquitetura	7
6	Considerações finais	8

1 Introdução

1.1 Objetivo

Este documento reúne, na íntegra, o **código-fonte desenvolvido** para o *Sistema de Navegação Autônoma para o Robô Móvel Khepera IV*, conforme exigido na entrega final do projeto. As listagens são extraídas diretamente dos arquivos do repositório, garantindo fidelidade ao código efetivamente executado e avaliado no Protocolo de Testes.

1.2 Escopo

O software compreende três artefatos: (i) o **mundo de simulação** (cenário do Webots), (ii) o **controlador de navegação** em linguagem C e (iii) o **script de compilação** (*Makefile*). A lógica de navegação — arquitetura híbrida com máquina de estados — está descrita no documento de Arquitetura; aqui o foco é o código.

2 Organização e estrutura do código

A estrutura de diretórios segue a convenção de projetos do Webots (pastas **worlds/** e **controllers/**):

```
eng_soft_rt/
  worlds/
    first.wbt <- mundo de simulacao (cenario)
  controllers/
    goto_avoid/
      goto_avoid.c <- controlador de navegacao (FSM)
      Makefile <- script de compilacao
```

Tabela 1: Artefatos de código e métricas.

Arquivo	Linguagem	Linhas	Descrição
worlds/first.wbt	VRML/PROTO (Webots)	83	Arena, obstáculos, alvo e robô (com GPS+IMU).
controllers/goto_avoid/goto_avoid.c	C	163	Controlador de navegação com máquina de estados.
controllers/goto_avoid/Makefile	Make	6	Compilação via cadeia de ferramentas do Webots.
Total	—	252	—

3 Compilação e execução

O controlador em C é compilado pela **cadeia de ferramentas embarcada no Webots** (MinGW-w64), sem dependências externas. Há duas formas:

- (a) **Pela interface gráfica:** abrir `worlds/first.wbt` no Webots e pressionar *Play*; o controlador é compilado automaticamente e a simulação inicia.
- (b) **Por linha de comando (execução *headless*, usada nos testes):**

```
set WEBOTS_HOME=C:\Program Files\Webots
make -C controllers\goto_avoid # compila goto_avoid.exe
webots --batch --mode=fast --no-rendering worlds\first.wbt
```

A telemetria periódica é gravada em `controllers/goto_avoid/run.log`, permitindo a verificação objetiva dos casos de teste.

4 Listagens

4.1 Mundo de simulação — `worlds/first.wbt`

Define o sistema de coordenadas, a arena 3×3 m, os cinco obstáculos, o marcador de alvo e a instância do robô Khepera IV, equipada com GPS e unidade inercial no *turret slot* e associada ao controlador `goto_avoid`.

```
1 #VRML_SIM R2025a utf8
2
3 EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/backgrounds/
4   protos/TexturedBackground.proto"
5 EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/backgrounds/
6   protos/TexturedBackgroundLight.proto"
7 EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/floors/protos/
8   RectangleArena.proto"
9 EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/factory/
10  containers/protos/WoodenBox.proto"
11 EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/robots/k-team/khepera4/
12  protos/Khepera4.proto"
13
14 WorldInfo {
15   basicTimeStep 16
16   contactProperties [
17     ContactProperties {
18       material2 "khepera4 caster wheel"
19       coulombFriction [
20         0.01
21       ]
22     }
23   ]
24 }
25
26 Viewpoint {
27   orientation -0.5773 0.5773 0.5773 2.0944
28   position 0 0 4.6
29 }
30
31 TexturedBackground {
32 }
33
34 TexturedBackgroundLight {
35 }
36
37 RectangleArena {
38   floorSize 3 3
39   wallHeight 0.15
40 }
41
42 # ---- ALVO (apenas visual: disco vermelho marcando o ponto B) ----
43 DEF GOAL Solid {
44   translation 1.0 1.0 0.001
45   children [
46     Shape {
47       appearance PBRAppearance {
48         baseColor 1 0 0
49         metalness 0
50         roughness 1
51       }
52       geometry Box {
53         size 0.18 0.18 0.002
54       }
55     }
56   ]
57 }
```

```

46   ]
47   name "goal"
48 }
49 # ---- OBSTACULOS (slalom na diagonal start -> goal) ----
50 WoodenBox {
51     translation -0.5 -0.5 0.1
52     size 0.2 0.2 0.2
53 }
54 WoodenBox {
55     translation 0 0 0.1
56     name "wooden box(1)"
57     size 0.25 0.25 0.2
58 }
59 WoodenBox {
60     translation 0.5 0.5 0.1
61     name "wooden box(2)"
62     size 0.2 0.2 0.2
63 }
64 WoodenBox {
65     translation 0.4 -0.3 0.1
66     name "wooden box(3)"
67     size 0.15 0.15 0.2
68 }
69 WoodenBox {
70     translation -0.3 0.4 0.1
71     name "wooden box(4)"
72     size 0.15 0.15 0.2
73 }
74 # ---- O ROBO: Khepera IV no ponto A (-1,-1), de frente pro alvo ----
75 Khepera4 {
76     translation -1.0 -1.0 0
77     rotation 0 0 1 0.785398
78     controller "goto_avoid"
79     turretSlot [
80         GPS {
81         }
82         InertialUnit {
83         }
84     ]
85 }

```

4.2 Controlador de navegação — controllers/goto_avoid/goto_avoid.c

Implementa o laço de controle periódico e a máquina de estados (NAVEGAR/GIRAR/CONTORNAR), incluindo percepção (GPS, IMU, IR), leis de controle, cinemática diferencial e telemetria.

```

1  /*
2   * goto_avoid.c -Khepera IV: navegacao ponto-a-ponto com desvio de obstaculos
3   * -----
4   * Disciplina: Engenharia de Software de Tempo Real (CETELI/UFAM)
5   *
6   * v2 -MAQUINA DE ESTADOS (corrige o "minimo local" da v1).
7   *
8   * Campo potencial puro TRAVA quando o alvo fica atras de um obstaculo
9   * (a forca que puxa anula a que empurra). A cura classica e PARAR de ser
10  * 100% reativo e se COMPROMETER com um contorno (ideia dos algoritmos Bug
11  * e do wall-following). Por isso 3 estados:
12  *
13  * NAVEGAR -> vai pro alvo (controle P de rumo) + desvio reativo leve
14  * GIRAR -> parede na frente: trava 1 direcao e gira ate limpar
15  * CONTORNAR-> anda reto uma distancia fixa p/ passar a caixa, e re-mira
16  *
17  * Tudo num laço periodico (wb_robot_step) = tarefa periodica de tempo real.
18  */
19
20 #include <webots/robot.h>
21 #include <webots/motor.h>
22 #include <webots/distance_sensor.h>

```

```

23 #include <webots/gps.h>
24 #include <webots/inertial_unit.h>
25
26 #include <stdio.h>
27 #include <math.h>
28
29 #define MAX_SPEED 47.6 /* rad/s -velocidade max das rodas */
30 #define CRUISE (0.25 * MAX_SPEED) /* velocidade de cruzeiro (~0.25 m/s) */
31
32 #define GOAL_X 1.0 /* ponto B (bate com o disco vermelho) */
33 #define GOAL_Y 1.0
34 #define GOAL_TOL 0.07 /* 7 cm: "chegou" */
35
36 /* limiares calibrados pela MEDICAO real (piso reflete -> base ~90-120): */
37 #define IR_BASE 100.0 /* leitura de base em espaco aberto */
38 #define IR_TRIG 210.0 /* >210: obstaculo de verdade -> GIRAR */
39 #define IR_CLEAR 160.0 /* <160: frente limpou (volta p/ base) */
40
41 #define K_HEADING 3.0 /* ganho P do rumo (go-to-goal) */
42 #define K_AVOID 0.05 /* ganho do desvio reativo */
43 #define SPIN 6.0 /* rad/s p/ girar no lugar no estado GIRAR */
44 #define PASS_DIST 0.30 /* m a percorrer no estado CONTORNAR */
45
46 typedef enum { NAVEGAR, GIRAR, CONTORNAR } State;
47
48 static double clampd(double v, double lo, double hi) {
49     return v < lo ? lo : (v > hi ? hi : v);
50 }
51
52 static double wrap_pi(double a) {
53     while (a > M_PI) a -= 2.0 * M_PI;
54     while (a < -M_PI) a += 2.0 * M_PI;
55     return a;
56 }
57
58 static double maxd(double a, double b) { return a > b ? a : b; }
59
60 int main(void) {
61     wb_robot_init();
62     setvbuf(stdout, NULL, _IONBF, 0); /* sem buffer: telemetria sai na hora */
63     FILE *logf = fopen("U:\\ufam\\eng_soft_rt\\controllers\\goto_avoid\\run.log", "w");
64     const int dt = (int)wb_robot_get_basic_time_step();
65
66     /* --- atuadores: rodas em modo VELOCIDADE --- */
67     WbDeviceTag left = wb_robot_get_device("left wheel motor");
68     WbDeviceTag right = wb_robot_get_device("right wheel motor");
69     wb_motor_set_position(left, INFINITY);
70     wb_motor_set_position(right, INFINITY);
71     wb_motor_set_velocity(left, 0.0);
72     wb_motor_set_velocity(right, 0.0);
73
74     /* --- sensores IR do anel --- */
75     const char *ir_names[8] = {
76         "rear left infrared sensor", "left infrared sensor",
77         "front left infrared sensor", "front infrared sensor",
78         "front right infrared sensor", "right infrared sensor",
79         "rear right infrared sensor", "rear infrared sensor"};
80     WbDeviceTag ir[8];
81     for (int i = 0; i < 8; i++) {
82         ir[i] = wb_robot_get_device(ir_names[i]);
83         wb_distance_sensor_enable(ir[i], dt);
84     }
85
86     /* --- localizacao: GPS + IMU --- */
87     WbDeviceTag gps = wb_robot_get_device("gps");

```

```

86  wb_gps_enable(gps, dt);
87  WbDeviceTag imu = wb_robot_get_device("inertial unit");
88  wb_inertial_unit_enable(imu, dt);
89
90  State state = NAVEGAR;
91  int escape_dir = 1; /* +1 = gira p/ esquerda, -1 = p/ direita */
92  double pass_x = 0, pass_y = 0;
93  const char *snames[3] = {"NAV", "GIR", "CON"};
94
95  printf("[goto_avoid v2] objetivo: ir ate (%.2f, %.2f)\n", GOAL_X, GOAL_Y);
96  if (logf) { fprintf(logf, "[goto_avoid v2] objetivo: ir ate (%.2f, %.2f)\n", GOAL_X, GOAL_Y);
97      fflush(logf); }
98  double last_print = -1.0;
99
100 /* ===== LACO DE CONTROLE ===== */
101 while (wb_robot_step(dt) != -1) {
102
103     /* ---- (1) PERCEPCAO ---- */
104     const double *pos = wb_gps_get_values(gps);
105     double yaw = wb_inertial_unit_get_roll_pitch_yaw(imu)[2];
106     double dx = GOAL_X - pos[0], dy = GOAL_Y - pos[1];
107     double dist = sqrt(dx * dx + dy * dy);
108
109     if (dist < GOAL_TOL) { /* chegou -> para tudo */
110         wb_motor_set_velocity(left, 0.0);
111         wb_motor_set_velocity(right, 0.0);
112         printf(">> CHEGUEI em (%.2f, %.2f) | t = %.1f s\n", pos[0], pos[1], wb_robot_get_time());
113         if (logf) { fprintf(logf, ">> CHEGUEI em (%.2f, %.2f) | t = %.1f s\n", pos[0], pos[1],
114             wb_robot_get_time()); fflush(logf); fclose(logf); logf = NULL; }
115         break;
116     }
117
118     double f = wb_distance_sensor_get_value(ir[3]); /* frente */
119     double fl = wb_distance_sensor_get_value(ir[2]); /* frente-esq */
120     double fr = wb_distance_sensor_get_value(ir[4]); /* frente-dir */
121     double l = wb_distance_sensor_get_value(ir[1]); /* esquerda */
122     double r = wb_distance_sensor_get_value(ir[5]); /* direita */
123     double block_l = clampd((fl - IR_BASE) + 0.5 * (l - IR_BASE), 0, 1e9);
124     double block_r = clampd((fr - IR_BASE) + 0.5 * (r - IR_BASE), 0, 1e9);
125     double front = maxd(f, maxd(fl, fr)); /* "tem parede a frente?" */
126
127     double goal_dir = atan2(dy, dx);
128     double err = wrap_pi(goal_dir - yaw);
129
130     /* ---- (2) MAQUINA DE ESTADOS ---- */
131     double forward = 0.0, steer = 0.0;
132     switch (state) {
133     case NAVEGAR:
134         /* vai pro alvo (P de rumo) + desvio reativo leve */
135         steer = K_HEADING * err + K_AVOID * (block_r - block_l);
136         forward = CRUISE;
137         forward *= (1.0 - 0.7 * clampd((f - IR_BASE) / (IR_TRIG - IR_BASE), 0, 1));
138         forward *= clampd(1.0 - 0.5 * fabs(err), 0.3, 1.0);
139         if (front > IR_TRIG) { /* parede! decide UMA vez e trava */
140             escape_dir = (block_l <= block_r) ? +1 : -1; /* gira p/ o lado livre */
141             state = GIRAR;
142         }
143         break;
144     case GIRAR:
145         /* gira no lugar, MESMA direcao, ate a frente limpar de verdade */
146         forward = 0.0;

```

```

147     steer = escape_dir * SPIN;
148     if (f < IR_CLEAR && fl < IR_CLEAR && fr < IR_CLEAR) {
149         pass_x = pos[0];
150         pass_y = pos[1];
151         state = CONTORNAR;
152     }
153     break;
154
155 case CONTORNAR:
156     /* anda reto p/ passar a caixa; so desvio leve; re-gira se topa outra */
157     forward = 0.7 * CRUISE;
158     steer = K_AVOID * (block_r - block_l);
159     if (front > IR_TRIG) {
160         escape_dir = (block_l <= block_r) ? +1 : -1;
161         state = GIRAR;
162     } else {
163         double tx = pos[0] - pass_x, ty = pos[1] - pass_y;
164         if (sqrt(tx * tx + ty * ty) > PASS_DIST)
165             state = NAVEGAR; /* contornou -> re-mira no alvo */
166     }
167     break;
168 }
169
170 /* ---- (3) CINEMATICA DIFERENCIAL ---- */
171 double vl = clampd(forward - steer, -MAX_SPEED, MAX_SPEED);
172 double vr = clampd(forward + steer, -MAX_SPEED, MAX_SPEED);
173 wb_motor_set_velocity(left, vl);
174 wb_motor_set_velocity(right, vr);
175
176 /* ---- telemetria (1x/s) ---- */
177 double t = wb_robot_get_time();
178 if (t - last_print >= 1.0) {
179     last_print = t;
180     printf("[%s] t=%4.1fs pos=(%.2f,%.2f) dist=%.2f err=%.2f IR[L/F/R]=%3.0f/%3.0f/%3.0f\n",
181           snames[state], t, pos[0], pos[1], dist, err, fl, f, fr, vl, vr);
182     if (logf) { fprintf(logf, "[%s] t=%4.1fs pos=(%.2f,%.2f) dist=%.2f err=%.2f IR[L/F/R]=%3.0f/%3.0f/%3.0f rodas=%.1f/%.1f\n",
183                   snames[state], t, pos[0], pos[1], dist, err, fl, f, fr, vl, vr); fflush(logf); }
184 }
185 }
186
187 wb_robot_cleanup();
188 return 0;
189 }

```

4.3 Script de compilação — controllers/goto_avoid/Makefile

```

1 # Webots Makefile system
2 # Compila automaticamente o controlador em C usando o toolchain embutido no Webots.
3
4 null :=
5 space := $(null) $(null)
6 WEBOTS_HOME_PATH=$(subst $(space),\ ,$(strip $(subst \,/,$(WEBOTS_HOME))))
7 include $(WEBOTS_HOME_PATH)/resources/Makefile.include

```

5 Correspondência código ↔ arquitetura

A Tabela 2 relaciona os principais trechos do controlador aos componentes da arquitetura e aos requisitos atendidos, facilitando a leitura e a avaliação.

Tabela 2: Mapa do código para componentes e requisitos.

Trecho do código	Componente	Requisitos
<code>wb_gps_get_values, wb_inertial_unit_*</code>	Percepção/Localização	RF-04
Leitura dos IR; <code>block_l, block_r</code>	Percepção de obstáculos	RF-02
<code>switch(state)</code> (Navegar/Girar/Contornar)	Arbitragem (FSM)	RF-02, RF-06
<code>steer = K_HEADING*err + K_AVOID*...</code>	Controle (lei de rumo)	RF-01, RF-05
<code>vl, vr, wb_motor_set_velocity</code>	Atuação (cinemática diferencial)	RF-05
<code>if (dist < GOAL_TOL)</code>	Detecção de chegada/parada	RF-03
<code>fprintf(logf, ...), fflush</code>	Telemetria	RF-07
<code>while (wb_robot_step(dt) != -1)</code>	Laço periódico (tempo real)	RT-01

6 Considerações finais

O código apresentado é o artefato verificado no Protocolo de Testes, no qual o sistema foi **aprovado** (missão concluída sem colisões, com escape de mínimo local e respeito ao período do laço). Os parâmetros de controle estão centralizados como constantes no início do controlador, favorecendo a manutenibilidade (RNF-05) e a futura portabilidade para o robô físico (RNF-04), bastando substituir a camada de localização (GPS→odometria).